UNIVERSIDAD TECNOLÓGICA DE PEREIRA
FACULTAD DE INGENIERÍAS
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
PROGRAMACIÓN III

Actividad 02 – Backtracking y SLD

Integrante: Esteban Guevara Cardona

Enlace Github: https://github.com/EstebanGC222/Prog-3---Actividad-02-Backtracking-y-SLD

1. Puentes de Königsberg

Las cuatro zonas de tierra son norte y sur (las orillas), kneiphof (la isla central) y este (la otra isla).

Programa

```
% Zonas: norte (orilla norte), sur (orilla sur),
%        kneiphof (isla central), este (isla este)
% puente(Id, Zona1, Zona2)
puente(p1, norte, kneiphof).
puente(p2, norte, kneiphof).
puente(p3, sur, kneiphof).
puente(p4, sur, kneiphof).
puente(p5, norte, este).
puente(p6, sur, este).
puente(p7, kneiphof, este).

% Los puentes se pueden cruzar en ambos sentidos
cruza(P, X, Y) :- puente(P, X, Y).
cruza(P, X, Y) :- puente(P, Y, X).

% Recorrido que usa cada puente una sola vez (backtracking)
recorrido(X, X, Usados, Usados).
recorrido(X, Fin, Usados, Total) :-
    cruza(P, X, Y),
    \+ member(P, Usados),
    recorrido(Y, Fin, [P|Usados], Total).

% Paseo: cruza los 7 puentes y vuelve al punto de partida
paseo(Inicio) :-
    member(Inicio, [norte, sur, kneiphof, este]),
    recorrido(Inicio, Inicio, [], Usados),
    length(Usados, 7).

% Grado: cantidad de puentes que llegan a una zona
grado(Zona, N) :-
    member(Zona, [norte, sur, kneiphof, este]),
    findall(P, cruza(P, Zona, _), L),
    length(L, N).
```

Consultas

```
?- paseo(X).
false.

?- grado(Zona, N).
```

```
Zona = norte, N = 3 ;
Zona = sur, N = 3 ;
Zona = kneiphof, N = 5 ;
Zona = este, N = 3.
```

Respuesta: no es posible. Prolog prueba por backtracking todos los recorridos desde cada zona y ninguno cruza los 7 puentes una sola vez volviendo al inicio. Coincide con el resultado de Euler: para que exista ese paseo, todas las zonas deben tener un número par de puentes, y aquí las cuatro tienen un número impar.

2. Rutas más cortas

Programa

```
% carretera(Ciudad1, Ciudad2, Km)
carretera(coruna, vigo, 171).
carretera(coruna, valladolid, 455).
carretera(vigo, valladolid, 356).
carretera(oviedo, bilbao, 304).
carretera(valladolid, bilbao, 280).
carretera(valladolid, madrid, 193).
carretera(bilbao, madrid, 395).
carretera(bilbao, zaragoza, 324).
carretera(madrid, zaragoza, 325).
carretera(zaragoza, barcelona, 296).
carretera(barcelona, gerona, 100).
carretera(barcelona, valencia, 349).
carretera(madrid, badajoz, 403).
carretera(madrid, jaen, 335).
carretera(madrid, albacete, 251).
carretera(albacete, valencia, 191).
carretera(albacete, murcia, 150).
carretera(valencia, murcia, 241).
carretera(jaen, sevilla, 242).
carretera(jaen, granada, 99).
carretera(sevilla, granada, 256).
carretera(sevilla, cadiz, 125).
carretera(granada, murcia, 278).

% Las carreteras son de doble sentido
conectado(X, Y, D) :- carretera(X, Y, D).
conectado(X, Y, D) :- carretera(Y, X, D).

% Camino de X a Y sin repetir ciudades (backtracking)
camino(X, Y, Ruta, Km) :- camino(X, Y, [X], R, Km), reverse(R, Ruta).
camino(X, X, Visitados, Visitados, 0).
camino(X, Y, Visitados, Ruta, Km) :-
    conectado(X, Z, D),
    \+ member(Z, Visitados),
    camino(Z, Y, [Z|Visitados], Ruta, K),
    Km is D + K.

% Ruta más corta: el camino de menor distancia
ruta_mas_corta(X, Y, Ruta, Km) :-
    findall(K-R, camino(X, Y, R, K), Caminos),
    keysort(Caminos, [Km-Ruta|_]).
```

Consultas

```
?- ruta_mas_corta(madrid, gerona, Ruta, Km).
Ruta = [madrid, zaragoza, barcelona, gerona], Km = 721.

?- ruta_mas_corta(coruna, cadiz, Ruta, Km).
Ruta = [coruna, valladolid, madrid, jaen, sevilla, cadiz], Km = 1350.

?- ruta_mas_corta(oviedo, murcia, Ruta, Km).
Ruta = [oviedo, bilbao, madrid, albacete, murcia], Km = 1100.
```

3. Grafo dirigido

Programa

```
% arista(Origen, Destino, Costo)  -- grafo dirigido
arista(vancouver, edmonton, 16).
arista(vancouver, calgary, 13).
arista(edmonton, saskatoon, 12).
arista(calgary, edmonton, 4).
arista(calgary, regina, 14).
arista(saskatoon, calgary, 9).
arista(saskatoon, winnipeg, 20).
arista(regina, saskatoon, 7).
arista(regina, winnipeg, 4).

% Conexión directa entre dos nodos, en cualquier sentido
conexion(X, Y, C) :- arista(X, Y, C).
conexion(X, Y, C) :- arista(Y, X, C).

% Un nodo tiene aristas si sale o llega alguna arista a él
tiene_aristas(X) :- (arista(X, _, _) ; arista(_, X, _)), !.

% Costo de ir de X a Z pasando por Y
costo(X, Y, Z, C) :-
    arista(X, Y, C1),
    arista(Y, Z, C2),
    C is C1 + C2.

% ¿Es posible viajar de X a Y? (sin repetir nodos, evita ciclos)
viajar(X, Y, Ruta, Costo) :- viajar(X, Y, [X], R, Costo), reverse(R, Ruta).
viajar(X, X, Visitados, Visitados, 0).
viajar(X, Y, Visitados, Ruta, Costo) :-
    arista(X, Z, C),
    \+ member(Z, Visitados),
    viajar(Z, Y, [Z|Visitados], Ruta, C2),
    Costo is C + C2.
```

Consultas

¿Existe una conexión entre Saskatoon y Vancouver?

```
?- conexion(saskatoon, vancouver, C).
false.

?- viajar(saskatoon, vancouver, Ruta, Costo).
false.
```

No hay arista directa entre ellos y tampoco un camino desde Saskatoon, porque ninguna arista llega a Vancouver.

¿Con qué nodos está conectado Regina y cuál es el costo de cada conexión?

```
?- conexion(regina, X, Costo).
X = saskatoon, Costo = 7 ;
X = winnipeg, Costo = 4 ;
X = calgary, Costo = 14.
```

Regla para determinar si un nodo tiene aristas:

```
?- tiene_aristas(regina).
true.

?- tiene_aristas(bogota).
false.
```

Regla para el costo de ir de un nodo X a un Z pasando por Y:

```
?- costo(vancouver, calgary, regina, C).
C = 27.

?- costo(vancouver, Y, saskatoon, C).
Y = edmonton, C = 28.
```

¿Es posible viajar desde Edmonton a Calgary?

```
?- viajar(edmonton, calgary, Ruta, Costo).
Ruta = [edmonton, saskatoon, calgary], Costo = 21.
```

Sí. No hay arista directa de Edmonton a Calgary (la que existe va de Calgary a Edmonton), pero se puede llegar pasando por Saskatoon, con un costo de 21.